IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.Doi Number

A Dual-Perspective Self-Supervised IoT Intrusion Detection Method Based on Topology Reconstruction and Feature Perturbation

RUISHENG LI[1], HUIMIN SHEN[2], QILONG ZHANG[2]

[1]School of Artificial Intelligence, Gansu University of Political Science and Law, Lanzhou 730070, China
[2]School of Cyberspace Security, Gansu University of Political Science and Law, Lanzhou 730070, China

Corresponding author: Huimin Shen (shenhuimin@stu.gsupl.edu.cn).

This work was supported in part by the 2024 Higher Education Industry Support Program under Grant CYZC-2024-24 and the University-level Research and Innovation Project of Gansu University of Political Science and Law under Grant GZF2024XZD05.

ABSTRACT The rapid growth of the Internet of Things has driven intelligent advancements across various fields. However, the proliferation of IoT devices and diverse network interactions also introduces significant security challenges. As a critical technology for securing IoT, intrusion detection systems aim to identify potential threats by analyzing network traffic features. Yet, traditional models struggle to capture the complex topological structures in IoT environments, and their training often relies heavily on large amounts of labeled data, making them unsuitable for IoT settings where massive data is continually generated. This paper presents a graph-based framework leveraging self-supervised learning to address these challenges. This framework employs a graph encoder to capture topological information and generate graph embeddings, utilizing topology reconstruction and feature perturbation to create diverse loss graphs that enhance feature learning. Additionally, the custom adaptive cosine loss function dynamically adjusts the loss weights of different samples and utilizes cosine similarity to align the embeddings of real and predicted graphs, thereby maximizing mutual information between local embeddings of real graphs and global summaries. The model was evaluated on four public datasets, and the experimental results demonstrate that the proposed method significantly improves detection performance. It outperforms existing self-supervised graph neural network models, validating the applicability of this approach in settings with unlabeled data.

INDEX TERMS Intrusion detection, graph neural networks, self-supervised learning, EE-GraphSAGE.

I. INTRODUCTION

With ongoing technological advancements, the Internet of Things (IoT) has revolutionized the information industry, driving digital transformation across various sectors [1]. Applications in smart homes, healthcare, and intelligent transportation are increasingly integrated into daily life, providing more convenient and efficient services. However, the sheer number of IoT devices and their extensive interconnectivity pose significant security challenges, making IoT security a critical concern [2]. Network intrusion detection systems (NIDS), a key component of network security, are essential for defending against malicious attacks by monitoring and analyzing network traffic to detect anomalies and threats [3].

Current NIDS solutions predominantly rely on machine learning (ML) and deep learning (DL) techniques. ML-based NIDS are suitable for small-scale datasets, offering faster training speeds [4], [5], while DL-based NIDS leverage automatic feature extraction to efficiently handle large-scale data and detect complex, nonlinear attack patterns [6], [7]. However, both ML and DL models struggle to capture the complex topological structures in IoT environments, resulting in limited feature representation and inadequate adaptability to the multi-dimensional interactions inherent in IoT networks.

The introduction of Graph Neural Networks (GNNs) offers a new approach to intrusion detection. GNNs are neural networks designed to process graph-structured data, learning embeddings for individual nodes or entire graphs through information propagation between nodes and edges. IoT consists of diverse device nodes and complex connectivity, naturally exhibiting graph-structured characteristics. Therefore, GNNs are an ideal choice for IoT scenarios. Existing GNN-based IoT intrusion detection models can

overcome the limitations of traditional models in adapting to complex graph structures, effectively extracting relational features between IoT devices. However, the performance of these models heavily relies on the quality of labeled data. In practical scenarios, obtaining high-quality labeled data is costly and time-consuming, resulting in suboptimal performance when handling IoT environment data [8].

Researchers have introduced self-supervised learning into the intrusion detection domain to address these challenges [9], [10]. Self-supervised learning is an unsupervised learning method in which the model generates pseudo-labels from data to learn useful feature representations without relying on labeled data. It is well-suited for the sparse and heterogeneous data commonly encountered in IoT environments. Existing self-supervised intrusion detection models based on GNNs have achieved high detection performance. However, these models still exhibit certain limitations when handling anomalous graph structures. Many of these models enhance graph features from a single perspective, resulting in a limited representation of the information captured by the model [11].

Building upon this, we propose a dual-perspective self-supervised IoT intrusion detection model that leverages a graph encoder to capture the spatial topological features of the data, addressing the limitations of traditional NIDS models, which treat IoT traffic as independent, sequential data. It generates loss graphs from two perspectives — topology reconstruction and feature perturbation—thereby enhancing graph embeddings from multiple angles. A custom loss function is also defined to maximize the mutual information between the real graph and its global embedding. To validate the effectiveness of the model, we conducted experiments on four publicly available datasets. The results indicate that the model successfully addresses the aforementioned challenges and achieves superior detection performance.

In summary, the primary contributions of this paper are as follows:

- We propose a graph augmentation strategy based on self-supervised learning, employing topological reconstruction and feature perturbation to generate multi-perspective loss graphs. This approach enhances the model's capacity to capture topological structures and feature variations. Additionally, we introduce Edge-Enhanced GraphSAGE (EE-GraphSAGE) as the graph encoder to generate high-quality graph embeddings.
- We developed an adaptive cosine loss function integrating adaptive weighting with cosine similarity. This enhances the model's focus on high-confidence samples while mitigating noise from low-confidence ones.
- We conducted experiments on four publicly available datasets, demonstrating the effectiveness of the proposed model.

The structure of the paper is as follows: Section II discusses related research on IoT intrusion detection based on graph neural networks, Section III presents the dual-perspective self-supervised method, Section IV covers the experimental setup and results analysis, and Section V provides a conclusion.

II. RELATED WORK

A. Deep Learning-based IoT NIDS

In IoT security, deep learning-based intrusion detection models have proven effective in identifying and classifying attack patterns by performing in-depth analysis and modeling of traffic data features. Deore et al. [12] proposed a NIDS based on a deep LSTM model optimized with Chimp Chicken Swarm Optimization (ChCSO). Hanafi et al. [13] developed an IoT intrusion detection model by integrating an enhanced binary grey wolf optimization algorithm with LSTM. Zhou et al. [14] introduced a novel anomaly detection model that combines an optimized deep convolutional autoencoder (ODCAE) with bidirectional gated recurrent units (BiGRU), replacing conventional convolutions with dilated convolutions to improve feature extraction.

B. GNN-based IoT NIDS

DL-based models cannot fully capture the topological information and inter-node relationship features in the complex networks of IoT. As a result, many researchers have combined GNNs with classical deep learning models to enhance the ability of the model to represent IoT data at the topological level. Lin et al. [15] proposed a GNN-based IoT intrusion detection system that integrates global and local attention mechanisms with graph contrastive learning in an edge-based GNN model. Zhang et al. [16] introduced the EGAT-LSTM model for malicious traffic classification, combining GAT for node feature extraction with LSTM networks to capture temporal dependencies between nodes.

Moreover, numerous studies have focused on GNN-based models for intrusion detection, seeking to extract more comprehensive information from the intricate graph structures of IoT networks, while simultaneously reducing model complexity. Zhong et al. [17] proposed the Six-GraphSecurity model, which uses a six-layer GraphSAGE neural network to classify malicious activities in industrial IoT, achieving high detection accuracy. Zhang et al. [18] introduced a GNN-based intrusion detection method, TPEGraphSAGE, which utilizes an edge embedding generation algorithm for feature aggregation and transformation, followed by training a graph edge classifier with the transformed data. Marfo et al. [19] combined GraphSAGE and GAT networks, with GraphSAGE creating embeddings of network activities through data interactions and GAT assigning greater weights to the most critical interactions. Deng et al. [20] presented EMA-IDS. This novel GNN-based model uses multi-hop attention to effectively aggregate node features and combine them with edge features, improving detection accuracy while reducing model complexity.

C. Self-Supervised GNN-based IoT NIDS

Supervised GNN-based models effectively capture complex network topological features but are dependent on large amounts of labeled data. To mitigate this limitation, researchers have increasingly turned to self-supervised learning, utilizing graph data augmentation methods and loss function optimization to extract data representations without relying on labeled data.

1) GRAPH AUGMENTATION RESEARCH

In recent years, numerous graph augmentation methods have been proposed. You et al. [21] proposed several techniques, including node removal, edge perturbation, and subgraph generation, to maximize mutual information, demonstrating that appropriate augmentation strategies can enhance performance on downstream tasks. Similarly, MERIT [22] constructs improved graph views by randomly adding or removing edges. DGI [23] trains a graph encoder by maximizing the mutual information between node embeddings and global graph representations, thereby learning efficient node-level features. Furthermore, InfoGraph [24] extends this concept to the graph level, maximizing mutual information across the entire graph to achieve superior global representations. These methods highlight the potential of self-supervised learning in feature extraction for tasks like intrusion detection.

These self-supervised learning methods offer novel approaches for efficient feature extraction in unlabeled environments and have thus gradually been introduced into intrusion detection. Venturi et al. [25] introduced a novel NIDS using the Adversarial Regularized Graph Autoencoder (ARGA), which encodes topological and node feature information into compact latent representations through unsupervised learning. Liu et al. [26] proposed a similarity-based graph contrastive learning model (SimGCL), which generates enhanced views by modifying the graph structure and utilizes these views to improve the homogeneity of node representations.

They often rely on a single view for augmentation, limiting the diversity of the learned representations. Further research has explored combining multiple augmentation strategies to enhance feature learning. The MVGRL model [27] employs subgraph sampling and diffusion mechanisms to generate two distinct graph views, explicitly modeling the relationship between node representations and graph summaries. In contrast, Duan et al. [28] proposed a method where the same node augmentation technique generates two enhanced graphs for contrastive learning, preserving the integrity of the graph structure but limiting the diversity of the views. Shi et al. [29] introduced the Smooth Activation Graph (ENGAGE), which combines two distinct augmentation strategies: one for perturbing the graph structure and another for perturbing the feature information.

Despite significant advancements in graph data augmentation and self-supervised learning, the aforementioned studies still exhibit certain limitations. Research [21]-[24] predominantly focuses on individual graph augmentation techniques, such as node removal or edge perturbation, which fail to comprehensively capture the diversity and complexity of graphs. Works [25] and [26] aim to enhance feature diversity through multi-view augmentation, but are constrained by their reliance on specific augmentation methods, limiting the effective integration of both global and local graph features. Although studies [27] and [28] employ more diverse augmentation strategies, they do not adequately combine the multi-dimensional features of the graph, restricting their performance in complex scenarios. In contrast, our self-supervised model generates loss graphs from both topological and feature perspectives, improving the model's ability to comprehensively capture graph structure and attribute information, thereby overcoming the limitations of previous methods that depend on a single view or feature.

2) LOSS FUNCTION RESEARCH

As graph augmentation techniques have diversified, some studies have focused on improving loss functions to more effectively guide model learning. Nguyen et al. [30] proposed an intrusion detection method, TS-IDS, which combines node-supervised and edge-supervision losses to form a comprehensive loss function. CCA-SSG [31] introduced an additional loss function that encourages the cross-correlation matrix of two view representations to approximate the identity matrix.

Nonetheless, these methods exhibit certain limitations. Study [30] provides limited capabilities in handling noisy data, while the constraints in [31] primarily target the view level, neglecting the feature variations across samples. To overcome these challenges, we propose an adaptive cosine loss function that combines a weighting mechanism with cosine similarity loss, enabling dynamic weight assignment to samples. This approach effectively reduces the influence of noisy samples and enhances the model's sensitivity to feature differences.

III. Method

A. Data Preprocessing

Through dataset inspection, we identified that certain features (e.g., the ssl_cipher column in the ToN-IoT dataset, where all values were '-') were either missing or constant across all samples, rendering them ineffective for model training and therefore removed. Non-numeric features were encoded using label encoding, and missing or infinite values were filled with zero to maintain data integrity and consistency. All minority class samples were retained, while random non-replacement sampling was applied to the excessively large majority class to mitigate the impact of data imbalance on model detection performance. To ensure user data privacy, source IP addresses were randomized and paired with port numbers to maintain the anonymity and non-traceability of the IP sources, thereby preventing the disclosure of user IP information. To improve training efficiency, all features were normalized, and target labels were encoded in numerical format. These preprocessing steps refined the model inputs, enhancing their suitability for constructing graph-based data structures.

B. Graph Construction and Model Design

The experiment begins by transforming the preprocessed data into graph data, where the IP and traffic features represent the

node and edge information, respectively. The model generates local and global embeddings of the "real graph" and "loss graphs" through a graph encoder and uses a discriminator to differentiate between the embeddings of the real graph and the loss graphs. Finally, a custom loss function is applied further to optimize the feature representation capability of the graph encoder. Fig.1 illustrates the model architecture.

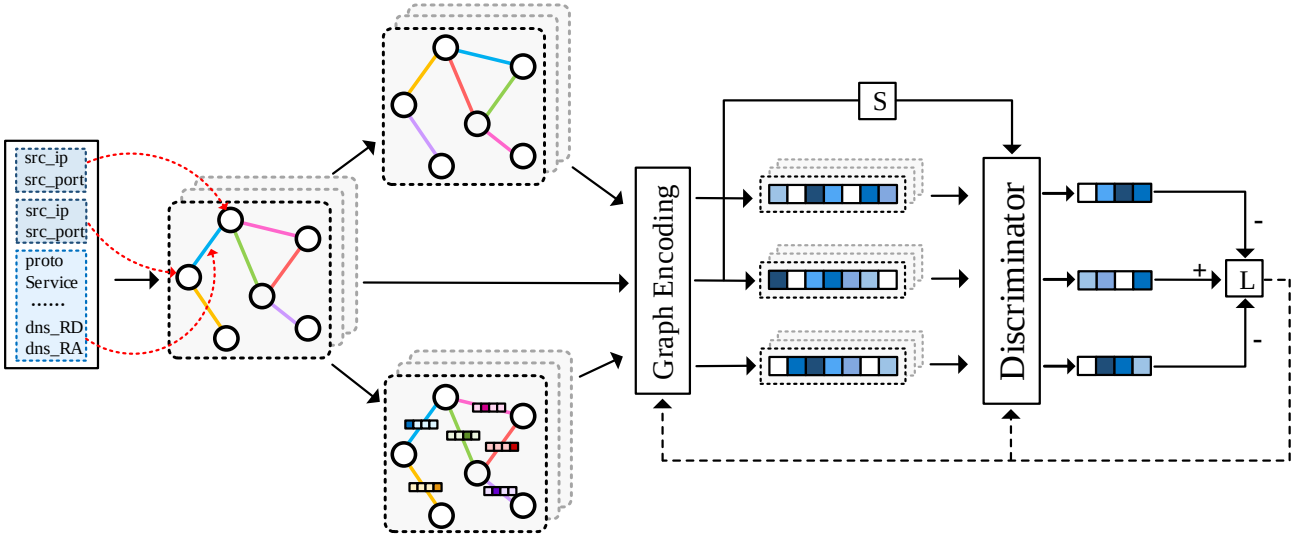


FIGURE 1. **Model architecture diagram.**

1) CONSTRUCTING GRAPH DATA

To convert traffic data into graph-structured data, this study models source and destination IP addresses as graph nodes, with other features representing the graph edges. A graph is constructed with source nodes represented by randomly mapped source IP addresses and destination nodes by destination IP addresses (with IP addresses encoded as string features). The remaining features (e.g., protocol type, packet count) are normalized and stored as edge features in a new column. For the NF series datasets without IP address features, source and destination ports are used as node features. The constructed multi-graph allows multiple edges between the same source and destination nodes (e.g., if two traffic flows share the same source and destination IPs, no new nodes are created, and the edges share the same source and destination nodes). This approach simplifies the graph structure compared to creating separate graphs for each flow, eliminating the need to re-establish connections between nodes. However, this approach for graph construction replaces the spatial topology information carried by the IP addresses with the graph's connectivity, making the IP address feature of the nodes serve only as an identifier without containing useful information. The primary feature information is placed on the edges. Accordingly, we assign effective initial features to the nodes by aggregating the features of adjacent edges. Specifically, the features of each node's adjacent edges are aggregated by averaging, and a linear transformation is applied to map the aggregated features into a new feature space, which is then assigned to the node. The dimensionality of the node features is consistent with that of the edge features, ensuring alignment between node and edge embeddings during subsequent model training, thereby improving training efficiency.

2) DUAL-PERSPECTIVE GRAPH AUGMENTATION INTRUSION DETECTION FRAMEWORK

The model first constructs a real graph based on the data and generates two loss graphs by performing topological reconstruction and feature perturbation on the real graph. These three graphs (one real graph and two loss graphs with different perspectives) are then input into the graph encoder, EE-GraphSAGE, to generate embeddings for each graph. Only the edge embeddings are selected as the local feature representation for each graph. The edge embeddings of the real graph are aggregated to produce a global representation, which serves as the unique global summary. Subsequently, a discriminator is used to evaluate the degree of match between the local embeddings of the three graphs and the global summary, scoring the embeddings of the graphs. Ideally, the local embedding of the real graph and the global summary should be highly similar in the feature space. Finally, a custom loss function, combining adaptive weighting and cosine similarity loss, is used to optimize the real graph's discriminator score to approach 1 while constraining the discriminator score of the loss graphs to approach 0, thereby training the graph encoder to effectively distinguish between the real and loss graphs.

We construct the preprocessed data into a graph structure and generate loss graphs based on this graph using two graph augmentation techniques: topology reconstruction and feature perturbation. The aim is to perturb the original graph data in terms of both spatial topology and edge features. Topology reconstruction refers to shuffling the order of edges in the original graph. Specifically, a random permutation index of all edges in the graph is generated, and the edge features are reordered according to this index. This introduces a perturbation to the edge sequence, while preserving the graph's

fundamental topological structure. This method simulates changes in the topology of real IoT networks, such as when devices like cameras in a smart home fail or lose power, potentially altering communication paths between devices. Topology reconstruction randomizes the order of edges to simulate these path adjustments, with the reordering of edges reflecting the dynamic changes in communication paths. Feature perturbation refers to the random disturbance of certain edge features in the graph. Specifically, 20% of the edges are randomly selected, and one feature from each selected edge is perturbed. The perturbation methods include mathematical operations such as squaring or taking the square root of the chosen feature values. These perturbations simulate real-world data uncertainty, such as the effects of wireless signal interference on sensor data transmission, which may cause feature value deviations and impact transmission accuracy. This approach reflects the data variations encountered during transmission in a real IoT environment. Both augmentation strategies account for changes in the graph's overall topology and alterations in its internal edge features, effectively improving the model's ability to adapt to topological and feature-based changes. Furthermore, neither topology reconstruction nor feature perturbation alters the total number of nodes or edges in the original graph, ensuring no additional data bias is introduced.

The generated real graph and two loss graphs are input into the EE-GraphSAGE graph encoder to generate graph embedding representations. EE-GraphSAGE is a graph neural network model that updates node features by aggregating the features of neighboring nodes and adjacent edges during the message-passing process. The updated node features are then passed to the edges for edge feature updating. This mechanism allows the edges to acquire feature information from two-hop neighbors during the message-passing process while assigning higher weights to the nearest neighboring edges. Fig.2 illustrates the entire process. After constructing the real graph and loss graphs, they are input into the graph encoder. First, node features are updated by aggregating the features of adjacent edges and neighboring nodes through averaging, with the updated node feature dimension increasing from 30 to 128. Subsequently, edge features are updated using the features of the source and destination nodes of each edge, with the updated edge feature

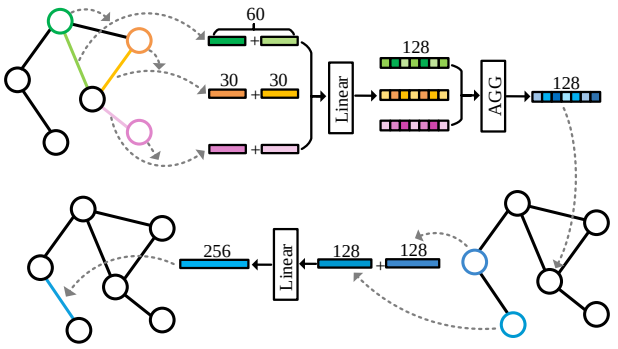


FIGURE 2. EE-GraphSAGE model framework diagram.

dimension expanding from 30 to 256. The stepwise increase in dimensionality provides a larger representation space, enabling node and edge features to capture more fine-grained information.

In the final output of node and edge embeddings, we use the edge embeddings, which contain critical information, as the local embedding representation of the graph. These embeddings serve as input to the discriminator in the subsequent step. We aggregate the edge embeddings of the real graph to generate the global embedding, which acts as the unique global summary S and serves as input to the discriminator. To create the global summary, we first compute the minimum value across each dimension of the edge embeddings in the real graph, extracting representative features. We then transform these minimum values non-linearly using the Sigmoid activation function, resulting in a 256-dimensional vector called the global summary S. The specific Eq. 1 is shown below, where "positive" refers to the local embedding of the real graph.

$$S = Sigmoid\big(min(positive)\big) \tag{1}$$

The discriminator primarily computes the similarity between the graph representations generated by the graph encoder and the global summary. It receives two inputs: the graph embedding representations and global summaries. These embeddings are mapped to a new space via a learnable weight matrix W, where the number of rows and columns of W matches the dimension of the global summary, which is 256, facilitating subsequent matrix multiplication. The discriminator then computes the similarity in two steps:(a) The graph embedding features are multiplied by the weight matrix W to generate weighted features. (b) The weighted features are multiplied by the global summary to produce the final score, representing the similarity between the input and real graphs. The higher the score, the closer the embedded features are to the original graph representation. The formula for calculating similarity is shown in Eq. 2, where W represents the weight matrix initialized by the discriminator, S denotes the global summary, *pos* represents the local embedding representation of the input graph, and *pos_score* is the similarity score output by the discriminator.

$$pos\_score_n = pos_{n\times 256} * W_{256\times 256} * S_{256} \tag{2}$$

The adaptive cosine loss function (ACos Loss) proposed in this paper improves upon the traditional binary cross-entropy (BCE) loss in two ways: by introducing an adaptive weighting mechanism and optimizing with cosine similarity loss.

The first improvement is the introduction of an adaptive weighting mechanism. Traditional BCE loss assigns equal weight to all samples, which can lead to overfitting on noisy or outlier samples. To mitigate this, we introduce an adaptive weighting mechanism that dynamically adjusts sample weights, enhancing the contribution of high-confidence samples. As shown in Eq. 3, when the predicted value p is close to the target value y, the term $\exp(-|p-y|)$ approaches

1, causing the adaptive weight to approach 2, assigning higher weights to high-confidence samples. Conversely, when the difference between p and y is large, $\exp(-|p-y|)$ approaches 0, and the adaptive weight approaches 1, thus reducing the impact of low-confidence samples. Multiplying the adaptive weight with the BCE loss effectively mitigates the impact of noisy data while giving higher weights to high-confidence samples (those with a small discrepancy between predicted and target values), thereby strengthening the model's ability to learn from high-confidence samples.

$$adaptive\_w = 1.0 + exp(-|p - y|) \quad (3)$$

The second improvement is incorporating cosine similarity loss. Cosine similarity loss measures the similarity between two vectors in terms of their directional alignment. The cosine similarity loss is calculated as 1 minus the cosine similarity. During model training, the goal is to make p and y more similar. As shown in Eq. 4, The expression $\frac{p \cdot y}{||p|| \cdot ||y||}$ represents the cosine similarity between p and y. As the similarity between p and y increases, the cosine similarity increases, resulting in a smaller cosine loss. Conversely, as the similarity decreases, the cosine similarity becomes smaller, leading to a larger cosine loss. By minimizing the cosine similarity loss, the model's predicted vector p is optimized to align more closely with the true label vector y, thus bringing it closer to the true label distribution.

$$cos\_loss = 1 - \frac{p \cdot y}{||p|| \cdot ||y||} \quad (4)$$

The final ACos Loss function, incorporating the improvements outlined above, is presented in Eq. 5:

$$ACos\ Loss = adaptive\_w * BCE_{(p,y)} + cos\_loss \quad (5)$$

The adaptive weighting mechanism, integrated with BCE loss, effectively amplifies the contribution of high-confidence samples, while the cosine similarity loss refines the alignment of the predicted vector with the true label vector in terms of direction.

We compute the loss for the real and loss graphs separately in the loss calculation process. For the real graph, the loss is calculated using the Acos Loss function to measure the discrepancy between the discriminative score and 1, thereby maximizing the mutual information between the local embedding of the real graph and the global summary. For the loss graph, we compute the discriminative scores for two different loss graphs and measure the discrepancy between these scores and 0, thus constraining the mutual information between the local embedding of the loss graph and the global summary. The final total loss is the sum of the losses from the three graphs. This approach enables the encoder to capture both global and local semantic relationships of the real graph, while also learning to identify and mitigate the impact of noisy or loss graphs. As shown in Eq. 6:

$$L_{total} = L_{pos} + L_{neg1} + L_{neg2} \quad (6)$$

In the equation, L_{pos} represents the loss of the real graph, while L_{neg1} and L_{neg2} correspond to the losses of two types of perturbed graphs. The total loss, L_{total}, captures both the global and local semantic associations of the real graph, while also enhancing the model's robustness and generalization capacity by mitigating the interference from the loss graphs.

3) MODEL PREDICTION

After training the model, we evaluate its performance using the test dataset. We construct the graph structure data for the test set and generate graph embedding representations through the trained graph encoder. The resulting embeddings are then input into a classifier based on the LightGBM (Light Gradient Boosting Machine) algorithm for classification. LightGBM uses a histogram-based algorithm for decision tree construction, discretizing continuous features into histograms, which reduces memory usage and computational complexity.

IV. Experiments and Results Analysis

A. Dataset

- **ToN-IoT**: The dataset, created by Abdullah et al. in 2019, is designed for IoT network traffic, simulating real-world IoT device usage, including operating system logs and remote sensing data from IoT/IIoT services. Its diverse device types and complex communication patterns make it a valuable resource for simulating realistic IoT scenarios and essential for evaluating the generalization capability of models in real IoT environments.
- **NF-BoT-IoT**: The dataset is specifically designed to assess intrusion detection systems in IoT environments, encompassing various attack types such as botnets and distributed denial-of-service (DDoS) attacks. It provides rich data on common IoT attacks, enabling research focused on evaluating the robustness and response capabilities of models to malicious threats, thereby enhancing the practical applicability of IoT intrusion detection systems.
- **NF-CSE-CIC-IDS2018-v2**: Developed by the Canadian Institute for Cybersecurity (CIC), this dataset is based on real network traffic and includes both normal traffic and various network attack scenarios, covering a wide range of attack types and traffic patterns. Its diversity and realism make it a high-quality benchmark dataset, offering a reliable platform for evaluating model performance in complex and dynamic network environments.
- **NF-UNSW-NB15-v2**: Proposed by UNSW, this dataset covers a broad spectrum of network protocols and operation types, providing a comprehensive collection of both attack and normal traffic samples. Its primary strength lies in the diversity of network protocols and attack scenarios, making it particularly well-suited for evaluating model performance across different network layers and protocols. This enables a deeper understanding of the model's ability to adapt to various complex attack types and protocols.

B. Experimental Environment Setup and Evaluation Metrics

The detailed information about the hardware and software environment used in the experiments is provided in Table 1. The model is trained using the Adam optimizer, which enables faster convergence and facilitates reaching the global optimum more efficiently. The specific experimental parameters are shown in Table 2.

TABLE 1. Hardware and software environment for experiments.

Hardware	CPU	Intel Xeon Platinum 8260L, 2.30GHz, 8 cores
	Memory	40 GB RAM
	Storage	Samsung SSD 870
	Motherboard	X11DPi-N
Software	Operating System	Ubuntu 20.04
	Python Version	Python 3.10
	DL Framework	PyTorch 2.0.1
	CUDA Version	CUDA 11.8

TABLE 2. Detailed parameter values.

Parameter Name	Value
EE-GraphSAGE Neural Network Layers	num_feature-128-256
Classifier Neural Network Layers	256-num_classes
Epoch	30
Learning Rate	0.001

This paper evaluates the model's performance based on accuracy, precision, recall, and F1 score, as shown in the following Eq. 7-10.

$$Accuracy(ACC) = \frac{TP + TN}{N} \tag{7}$$

$$Precision(p_c) = \frac{TP_c}{TP_c + FP_c} \tag{8}$$

$$Recall(R_c) = \frac{TP_c}{TP_c + FN_c} \tag{9}$$

$$F1 - score(F1_c) = \frac{2 \times p_c \times R_c}{p_c + R_c} \tag{10}$$

C. Experimental Results Analysis

1) DETECTION RESULTS OF THE MODEL ON THE TON-IOT DATASET

Table 3 presents the detection results for each class in the ToN-IoT dataset, showcasing the model's performance in detecting different classes.

The experimental results demonstrate that the proposed model performs exceptionally well across most attack types, with an overall weighted average accuracy of 0.9771, and recall and F1-scores close to 0.977. Most attack categories achieve accuracy above 0.95, with the model showing particularly strong performance in detecting Backdoor, Normal, and Ransomware attacks, where accuracy exceeds 0.98, highlighting its robust classification capability.

TABLE 3. Experimental results of the model on ToN-IoT dataset.

Attack	Accuracy	Precision	Recall	F1-score
Backdoor	0.9880	0.9986	0.9880	0.9933
DDoS	0.9692	0.9652	0.9693	0.9672
DoS	0.9466	0.9473	0.9466	0.9469
Injection	0.9172	0.9551	0.9172	0.9358
Mitm	0.7478	0.8600	0.7479	0.8000
Normal	0.9942	0.9748	0.9942	0.9844
Password	0.9650	0.9657	0.9650	0.9654
Ransomware	0.9804	0.9953	0.9804	0.9878
Scanning	0.9559	0.9944	0.9560	0.9748
XSS	0.9287	0.9353	0.9288	0.9320
W-avg	0.9773	0.9771	0.9773	0.9771

The model exhibits relatively weaker performance against MitM and Injection attacks, with accuracy and F1 scores of 0.7478 and 0.8000 for MitM attacks, and 0.9172 and 0.9358 for Injection attacks, respectively. MitM attacks typically involve intercepting and manipulating communication content, leading to significant changes in traffic features, some of which resemble normal traffic, complicating accurate classification. Injection attacks, which insert malicious code or commands, generate highly variable and complex traffic patterns, potentially contributing to the model's reduced detection efficiency. To address this, Generative Adversarial Networks (GANs) can be employed to augment the training dataset, particularly for stealthy and complex MitM and Injection attack samples, thereby improving the model's detection capabilities. Future work could explore leveraging deep learning techniques to extract higher-level features from traffic, with CNNs capturing local spatial features and LSTMs recognizing temporal patterns in attacks, further enhancing model performance.

2) PERFORMANCE EVALUATION OF THE SELF-SUPERVISED GRAPH ENCODING FRAMEWORK

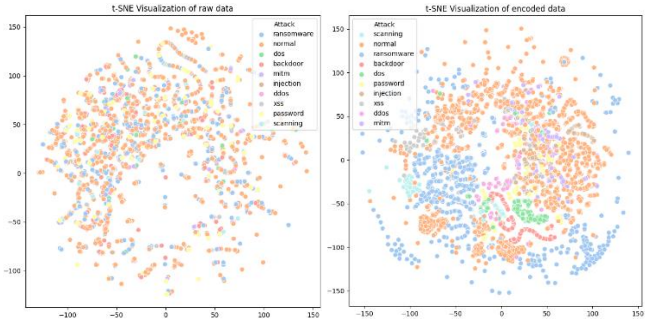
We compare our model with a supervised graph neural network approach. Our method employs the EE-GraphSAGE model as the graph encoder for training, which are subsequently used for classification. The supervised approach directly employs the EE-GraphSAGE graph neural network to train on labeled data and perform classification. Table 4 presents the classification results for both methods, while Fig.3 visualizes the data representations in a two-dimensional space before and after the graph encoding process.

TABLE 4. Detection performance comparison between EE-GraphSAGE and the proposed model.

Model	ACC	Pre	Rec	F1	epoch
EE-GraphSAGE	0.9013	0.9158	0.9013	0.9015	400
our	0.9773	0.9771	0.9773	0.9771	30

Table 4 compares the performance of the EE-GraphSAGE model under two different approaches. The supervised method achieves approximately 0.90 in accuracy, precision, recall, and F1-score. In contrast, the self-supervised method exceeds 0.97 in all these metrics. These results highlight that the self-supervised approach offers superior feature learning, outperforming the supervised graph neural network by 6-7

percentage points across multiple metrics. This demonstrates that self-supervised training effectively leverages unlabeled data to learn richer graph representations, improving classification performance. Furthermore, the self-supervised method significantly reduces training time, converging after just 30 epochs, far fewer than the 400 epochs required by the supervised method. This indicates that self-supervised learning accelerates model convergence through pretraining on unlabeled data, drastically shortening training time and offering significant advantages in resource-limited environments.



(a) Raw data (b) Data after graph encoding

FIGURE 3. t-SNE visualization of attack categories in the ToN-IoT dataset: (a) Raw data, (b) Data after graph encoding.

Fig.3 illustrates the significant differences in the data representations before and after processing with the graph encoder, as shown in the t-SNE plots. Fig. 3(a) displays the data before encoding, where the attack categories are scattered in the 2D space, resulting in a visualization where it is difficult to distinguish between them. In contrast, Fig. 3(b) shows the data after the trained graph encoder is processed, where the attack categories are clearly separated and exhibit more distinct clustering. This clustering suggests that the graph encoder has successfully captured the spatial and topological features of the data, effectively distinguishing between different attack categories. This demonstrates the encoder's effectiveness in handling complex graph-based data.

3) IMPACT OF DIFFERENT LOSS FUNCTIONS ON MODEL PERFORMANCE

Table 5 and Fig.4 compare the detection results of three different loss functions, including the ACos Loss function proposed in this paper, across various training epochs to validate the ACos Loss function's effectiveness. Fig.4 presents these results in the form of a line chart.

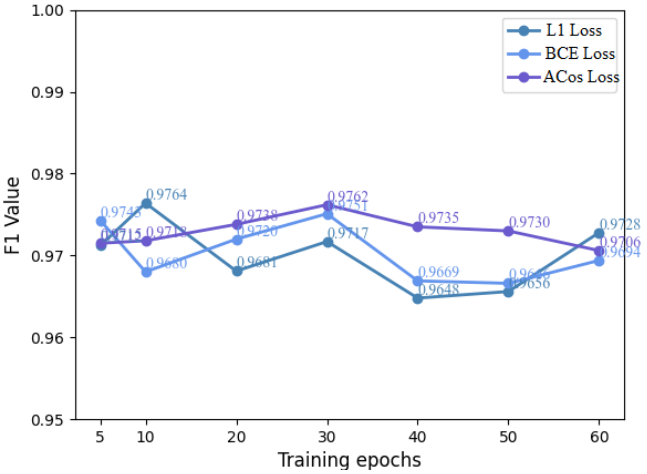


FIGURE 4. F1-Score comparison of different loss functions across training epochs.

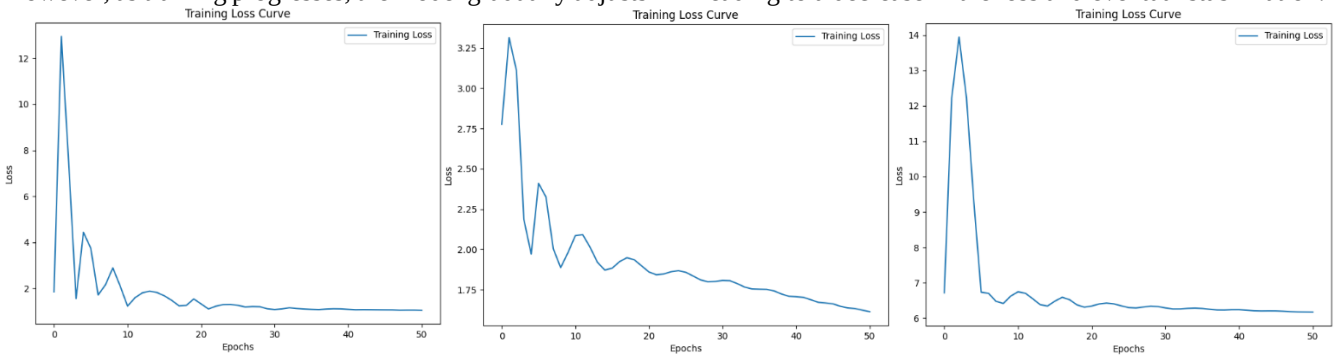
TABLE 5. Comparison of detection results with different loss functions.

	5	10	20	30	40	50	60
L1 Loss	0.9712	**0.9764**	0.9681	0.9717	0.9648	0.9656	**0.9728**
BCE Loss	**0.9743**	0.9680	0.9720	0.9751	0.9669	0.9666	0.9694
ACos Loss	0.9715	0.9718	**0.9738**	**0.9762**	**0.9735**	**0.9730**	0.9706

Table 5 shows the F1 scores of the model using different loss functions across various training epochs. Fig.4 presents these results in a more intuitive way, illustrating the impact of the three loss functions on model performance as training progresses. The F1 score of ACos Loss fluctuates minimally throughout the training process, peaking at 0.9762 in the 30th epoch, demonstrating excellent performance and higher resistance to overfitting. Additionally, ACos Loss maintains a consistently high F1 score throughout the training, staying above 0.97 as the epochs increase, in contrast to the slight decline observed in the BCE and L1 loss functions in the later stages of training. The experimental results indicate that the Acos Loss function exhibits higher stability and consistency across different training epochs, outperforming the other loss functions in the current task.

Fig.5 shows the loss curves generated by the model using three different loss functions over 50 training epochs. As seen in the figure, compared to L1 loss and BCE loss, the ACos Loss function proposed in this paper stabilizes more quickly, reaching a steady state after just 25 epochs. The ACos Loss function combines BCE loss and a cosine similarity regularization term. BCE loss helps to quickly reduce prediction errors in the early stages of training, prompting the model to adjust rapidly, while the cosine similarity term aligns the direction of the predicted values with the true labels, providing additional regularization. As a result, ACos Loss effectively controls the model's behavior at different training stages, leading to a more stable training process. The early loss spikes are likely due to the random initialization of the model's weights, causing a large initial difference between the predictions and true labels, which results in significant errors and fluctuations in the loss. Additionally, the exponential term $\exp(-|p-y|)$ in the loss function strongly penalizes large errors in the early stages of

training, potentially causing a temporary spike in the loss. However, as training progresses, the model gradually adjusts the weights, and the predictions move closer to the true labels, leading to a decrease in the loss and eventual stabilization.



(a) L1 Loss (b) BCE Loss (c) ACos Loss

FIGURE 5. **Training loss curves for different loss functions: (a) L1 loss, (b) BCE loss, (c) ACos Loss.**

TABLE 6. **Performance and lightweight comparison of five classifiers on ToN-IoT.**

Classifier	Acc	Pre	Rec	F1	Train(s)	Test(s)	Size(kb)
LightGBM	0.9746	0.9747	0.9746	0.9743	0.9197	0.6840	3479
KNN	0.9765	0.9766	0.9765	0.9764	0.8973	12.3931	94740
RF	0.9637	0.9637	0.9637	0.9637	0.8990	0.1601	7816
ET	0.9742	0.9744	0.9742	0.9741	0.8992	0.1599	17300
LR	0.7943	0.7439	0.7943	0.7478	0.9039	0.0939	22

4) DEPLOYABILITY ANALYSIS OF THE MODEL ON IOT DEVICES

During the model's prediction process, data is initially encoded through the graph encoder, and the encoded results are then input into the classifier for classification. The graph encoder has a model size of 294KB and an encoding time of 0.0011 seconds, demonstrating its lightweight and efficient characteristics. As such, the primary focus of the comparison is on the performance of the classifier in downstream tasks. Table 6 presents a comparison of the detection performance (accuracy, Precision, recall, F1-score, etc.) and lightweight requirements (model size, inference time, etc.) for five different classifiers on the ToN-IoT dataset, assessing the feasibility of deploying the model in IoT environments. The experimental results are shown in Table 6, where KNN, RF, ET, and LR refer to K-Nearest Neighbors, Random Forest, Extra Trees, and Logistic Regression, respectively.

The results indicate that while LR achieves optimal inference time (0.0939 seconds) and model size (22KB), its classification performance, with an F1 score of 0.7478, fails to meet the task requirements. In contrast, RF, with an inference time of 0.1601 seconds and a model size of 7816KB, is more lightweight than other methods, although its classification performance is slightly lower by 1%.

LightGBM, KNN, and ET exhibit similar classification performance, with F1 scores exceeding 0.9740. However, KNN's inference time is longer due to the need to compute distances between each test sample and all training samples. In the same experimental setup, KNN's inference time reaches a maximum of 12.29 seconds, with a model size of 92.5MB, making it unsuitable for resource-constrained IoT devices. LightGBM, with an F1 score of 0.9743, delivers the second-best detection performance, just 0.21% lower than KNN. Its smaller model size (3482KB) and faster inference time (0.622 seconds) make it well-suited for IoT environments, where high performance and low resource consumption are crucial, making it the most optimal choice.

We also implemented lightweight optimizations to the ET classifier, which exhibited a short inference time (0.1599 seconds) and strong classification performance (F1 = 0.9741), including weight quantization, depth restriction, and conversion to the ONNX format. While these optimizations significantly reduced the model size, they also resulted in a performance degradation (F1 dropped to 0.84 when the model size was reduced to 195KB), falling below the unoptimized LightGBM. Therefore, considering both detection performance and lightweight requirements, LightGBM was ultimately chosen as the classifier, with further lightweight optimizations applied.

LightGBM performs classification or regression tasks by constructing a series of decision trees. By tuning its hyperparameters, we can effectively control the model's complexity and computational resource usage. The parameter settings and corresponding detection results are presented in Table 7, where the parameters listed in the "Parameters" column are: n_estim, max_depth, num_leaves, and min_data_in_leaf.

The experimental results demonstrate that with parameter settings of 50-5-25-50, the model's inference time is reduced by 50% to 0.3214 seconds, while maintaining classification performance (F1 = 0.9771). Moreover, the model size is reduced to 1.3MB, achieving a balance between lightweight requirements and classification accuracy.

TABLE 7. LightGBM hyperparameter settings and detection results.

Parameter	Acc	Pre	Rec	F1	Train(s)	Test(s)	Size(kb)
100-(-1)-31-20	0.9746	0.9747	0.9746	0.9743	0.9197	0.684	3479
50-(-1)-31-20	0.9719	0.9716	0.9719	0.9712	0.9171	0.3558	1734
50-5-31-20	0.9730	0.9724	0.9730	0.9724	0.9186	0.3198	1526
50-3-31-20	0.9605	0.9596	0.9605	0.9590	0.9169	0.2253	505
50-5-25-20	0.9753	0.9746	0.9753	0.9745	0.9140	0.3122	1390
50-5-25-50	0.9773	0.9771	0.9773	0.9771	0.9167	0.3124	1340

The LightGBM model, based on decision trees, offers low computational complexity and minimal inference energy consumption, making it well-suited for low-power devices. With an overall model size under 2MB, it is compatible with most IoT devices, which often have limited memory and storage. Furthermore, the 0.3-second inference time meets the low-latency demands of real-time intrusion detection, making it particularly suitable for high-demand applications such as industrial control and smart homes.

5) COMPARISON WITH SUPERVISED MODELS

Table 8 presents the experimental results of our proposed self-supervised model compared to three supervised models-E-ResSAGE, EE-GraphSAGE, and Six-GraphS-on the ToN-IoT dataset.

TABLE 8. Comparison of classification performance between supervised learning and the proposed model on the ToN-IoT dataset.

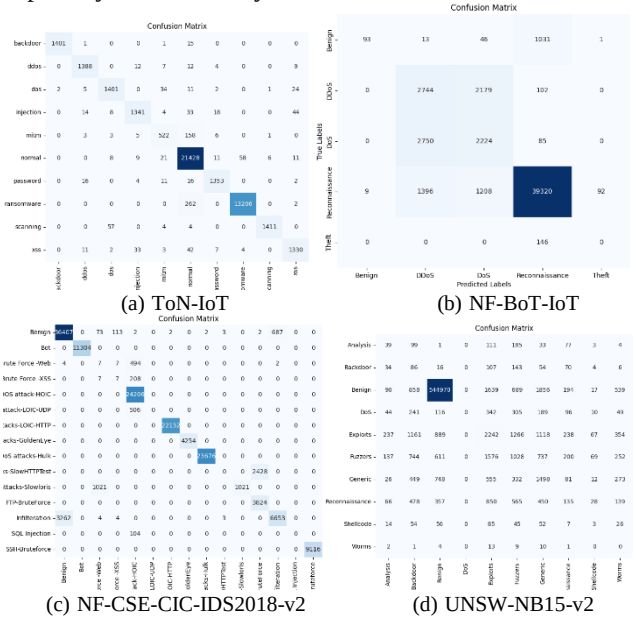
Model	Accuracy	Precision	Recall	F1-score
E-ResSAGE[32]	-	0.9221	0.9175	0.9190
EE-GraphSAGE	0.9013	0.9158	0.9013	0.9015
Six-GraphS[17]	0.8734	0.8841	0.8734	0.8701
our	0.9773	0.9771	0.9773	0.9771

The experimental results demonstrate that our model achieves accuracy, precision, recall, and F1-score exceeding 0.97. It outperforms the second-best E-ResSAGE model by 5.81% in terms of F1 score, surpassing the supervised graph neural network models used for comparison. This highlights that our model outperforms certain supervised intrusion detection models.

6) COMPARISON WITH SELF-SUPERVISED MODELS

We conducted experiments on multiple datasets. Fig.6 shows the confusion matrices of the model across four datasets. To validate the superiority of our proposed self-supervised model, we compared its performance with other graph neural network-based self-supervised intrusion detection models on these four datasets. The experimental results are presented in Table 9, demonstrating the superiority of our model.

The results in the table indicate that the proposed model achieves high accuracy and F1 scores on the NF-BoT-IoT, NF-CSE-CIC-IDS2018-v2, and NF-UNSW-NB15-v2 datasets. Specifically, it attains recall rates above 0.94 and F1 scores above 0.93 on both the NF-CSE-CIC-IDS2018-v2 and NF-UNSW-NB15-v2 datasets. While the F1 score on the NF-BoT-IoT dataset is slightly lower at 0.83, it still outperforms other self-supervised graph learning-based intrusion detection models. On the ToN-IoT dataset, the recall rate and F1 score of our model exceed those of the NEGSC method by 6.61% and 7.29%, respectively, demonstrating its superiority. These findings suggest that the proposed self-supervised model outperforms supervised models in intrusion detection tasks. Furthermore, it exhibits higher performance across all four datasets compared to other self-supervised models, highlighting its generalization capability and suitability for various IoT scenarios.



(a) ToN-IoT (b) NF-BoT-IoT

(c) NF-CSE-CIC-IDS2018-v2 (d) UNSW-NB15-v2

FIGURE 6. Confusion matrices of the model on different datasets. (a)ToN-IoT, (b)NF-BoT-IoT, (c)NF-CSE-CIC-IDS2018-v2, (d)UNSW-NB15-v2.

TABLE 9. Experimental results of ToN-IoT, NF-BoT-IoT, NF-CSE-CIC-IDS2018-v2, and NF-UNSW-NB15-v2 datasets under different models.

Data	Model	Rec	F1
ToN-IoT	Anomal-E	0.9693	0.9691
	NEGSC	0.9106	0.9044
	our	**0.9773**	**0.9771**
NF-BoT-IoT	Anomal-E[33]	0.8216	0.7847
	NEGSC[34]	0.8301	0.8270
	our	**0.8305**	**0.8335**
NF-CSE-CIC-IDS2018-v2	Anomal-E	0.8809	0.8598
	NEGSC	0.8720	0.8401
	our	**0.9410**	**0.9321**
NF-UNSW-NB15-v2	Anomal-E	**0.9628**	0.9559
	NEGSC	0.7334	0.8210
	our	0.9587	**0.9610**

V. Conclusion

This paper proposes a self-supervised IoT intrusion detection model to overcome the reliance on large labeled datasets in traditional intrusion detection systems. The model utilizes a

graph encoder to capture the spatial and topological features of the data. It generates two loss graphs through topological reconstruction and feature perturbation methods, which train the graph encoder to learn deeper data features. A custom adaptive cosine loss function optimizes the encoder's feature representations and improves the model's stability during training. This model outperforms existing approaches, especially in resource-constrained IoT environments. Experimental results on the ToN-IoT dataset show that the model achieves over 0.97 accuracy and F1-score. It also demonstrates strong classification performance on other IoT intrusion detection datasets.

Given the resource constraints in IoT environments, future research will focus on model optimization for such limitations. Knowledge distillation techniques, which transfer knowledge from complex models to simplified ones, can reduce computational burden while preserving accuracy, making them ideal for low-power devices and real-time intrusion detection. Furthermore, exploring additional graph enhancement strategies, such as optimizing graph structures or edge feature aggregation, will enhance the model's ability to identify complex network behaviors and attack patterns, addressing the increasing security challenges in IoT systems.

REFERENCES

[1] H. Nandanwar and R. Katarya, "Deep learning enabled intrusion detection system for Industrial IoT environment," *Expert Syst. Appl.*, vol. 249, part C, Sep. 2024, Art. no. 123808.

[2] S. I. Popoola, B. Adebisi, M. Hammoudeh, et al., "Hybrid deep learning for botnet attack detection in the Internet-of-Things networks," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4944–4956, Oct. 2020, DOI: 10.1109/JIOT.2020.3034156.

[3] Z. Gu, D. T. Lopez, L. Alrahis, et al., "Always be Pre-Training: Representation Learning for Network Intrusion Detection with GNNs," *Proc. 2024 25th Int. Symp. Quality Electronic Design (ISQED)*, 2024, pp. 1–8.

[4] C. Hazman, A. Guezzaz, S. Benkirane, and M. Azrour, "lIDS-SIoEL: Intrusion detection framework for IoT-based smart environments security using ensemble learning," *Cluster Comput.*, vol. 26, pp. 4069–4083, Nov. 2023.

[5] M. Al-Ambusaidi, Z. Yinjun, Y. Muhammad, and A. Yahya, "ML-IDS: An efficient ML-enabled intrusion detection system for securing IoT networks and applications," *Neural Networks*, vol. 28, pp. 1765–1784, Dec. 2023.

[6] X. Kan *et al*, "A novel IoT network intrusion detection approach based on adaptive particle swarm optimization convolutional neural network," *Inform. Sci.*, vol. 568, pp. 147–162, Aug. 2021. DOI: 10.1016/j.ins.2021.03.060.

[7] V. Saravanan, M. Madiajagan, S. M. Rafee *et al*, "IoT-based blockchain intrusion detection using optimized recurrent neural network," *Multimedia Tools Appl.*, vol. 83, pp. 31505–31526, Sep. 2023.

[8] F. Wei, H. Li, Z. Zhao, and H. Hu, "xNIDS: Explaining Deep Learning-based Network Intrusion Detection Systems for Active Intrusion Responses," *Proc. USENIX Security Symp. (USENIX Security 23)*, 2023, pp. 4337–4354.

[9] L. Deng, Y. Zhao, and H. Bao, "A Self-supervised Adversarial Learning Approach for Network Intrusion Detection System," in Proc. China Cyber Security Annual Conf., Singapore: Springer Nature Singapore, 2022, pp. 73–85.

[10] J. Zhang, Z. Shi, H. Wu, et al., "A Novel Self-supervised Few-shot Network Intrusion Detection Method," in Proc. Int. Conf. Wireless Algorithms, Systems, and Applications, Cham: Springer Nature Switzerland, 2022, pp. 513–525, DOI:10.1007/978-3-031-19208-1_42

[11] J. He, H. Kong, S. Zhang, et al., "Local Augmentation with Functionality-Preservation for Semi-Supervised Graph Intrusion Detection," in Proc. ICC 2024-IEEE Int. Conf. Communications, IEEE, 2024, pp. 2047–2052, DOI: 10.1109/ICC51166.2024.10622440

[12] B. Deore and S. Bhosale, "Hybrid optimization enabled robust CNN-LSTM technique for network intrusion detection," *IEEE Access*, vol. 10, pp. 65611–65622, Jun. 2022, DOI: 10.1109/ACCESS.2022.3183213.

[13] A. V. Hanafi, A. Ghaffari, H. Rezaei, et al., "Intrusion detection in Internet of Things using improved binary golden jackal optimization algorithm and LSTM," *Cluster Comput.*, vol. 27, pp. 2673–2690, Jul. 2023.

[14] Z. Zhou and Y. Ou, "Research on Network Anomaly Traffic Detection Based on ODCAE and BiGRU," in *Proc. 2023 IEEE 14th Int. Conf. Software Engineering and Service Science (ICSESS)*, Beijing, China, Dec. 2023, pp. 118–121.

[15] L. Lin, Q. Zhong, J. Qiu, et al., "E-GRACL: an IoT intrusion detection system based on graph neural networks," *J. Supercomput.*, vol. 81, art. no. 42, Oct. 2024, DOI: 10.1007/s11227-024-04778-x.

[16] L. Zhang, L. Tan, H. Shi, *et al*., "Malicious Traffic Classification for IoT based on Graph Attention Network and Long Short-Term Memory Network," in *Proc. 2023 24st Asia-Pacific Network Operations and Management Symp. (APNOMS)*, Daegu, South Korea, Sep. 2023, pp. 54–59.

[17] X. Zhong and G. Wan, "Six-GraphSecurity: Industrial Internet Intrusion Detection Based On Graph Neural Network," in *Proc. 2023 IEEE 7th Information Technology and Mechatronics Engineering Conf. (ITOEC)*, Chongqing, China, Dec. 2023, pp. 1340–1344.

[18] Y. Zhang, Y. Zhang, Y. Wu, *et al*., "TPE-NIDS: uses graph neural networks to detect malicious traffic," in *Proc. 2022 4th Int. Conf. Frontiers Technology of Information and Computer (ICFTIC)*, Chongqing, China, Oct. 2022, pp. 949–958.

[19] W. Marfo, D. K. Tosh, and S. V. Moore, "Enhancing Network Anomaly Detection Using Graph Neural Networks," in *Proc. 2024 22nd Mediterranean Communication and Computer Networking Conf. (MedComNet)*, Nafplio, Greece, Jun. 2024, pp. 1–10.

[20] P. Deng and Y. Huang, "Edge-featured multi-hop attention graph neural network for intrusion detection system," *Computers & Security*, vol. 148, Jan. 2025, Art. no. 104132. DOI: 10.1016/j.cose.2024.104132.

[21] Y. You, T. Chen, Y. Sui, et al. "Graph contrastive learning with augmentations," *Advances in neural information processing systems*, 2020, 33: 5812-5823.

[22] M. Jin, Y. Zheng, Y. F. Li, et al., "Multi-scale contrastive siamese networks for self-supervised graph representation learning," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 5812–5823, 2020. DOI: 10.48550/arXiv.2105.05682.

[23] P. Veličković, W. Fedus, W. L. Hamilton, *et al*., "Deep graph infomax," *arXiv preprint arXiv:1809.10341*, vol. 33, no. 2, pp. 1–17, Sep. 2018. Accessed on: Nov. 17, 2024. DOI: 10.48550/arXiv.1809.10341.

[24] F. Y. Sun, J. Hoffmann, V. Verma, *et al*., "Infograph: Unsupervised and semi-supervised graph-level representation learning via mutual information maximization," *arXiv preprint arXiv:1908.01000*, Aug. 2019. Accessed on: Nov. 17, 2024. DOI: 10.48550/arXiv.1908.01000.

[25] A. Venturi, M. Ferrari, M. Marchetti, *et al*., "ARGANIDS: A novel network intrusion detection system based on adversarially regularized graph autoencoder," *in Proc. 38th ACM/SIGAPP Symp. Applied Computing*, 2023, pp. 1540–1548.

[26] C. Liu, C. Yu, N. Gui, *et al*., "SimGCL: Graph contrastive learning by finding homophily in heterophily," *Knowledge and Information Systems*, vol. 66, no. 3, pp. 2089–2114, Nov. 2023.

[27] K. Hassani and A. H. Khasahmadi, "Contrastive multi-view representation learning on graphs," *in* Proc. Int. Conf. Machine Learning (ICML), PMLR, 2020, pp. 4116–4126.

[28] H. Duan, C. Xie, B. Li, et al., "Self-supervised contrastive graph representation with node and graph augmentation," *Neural Networks*, vol. 167, pp. 223–232, Oct. 2023.

[29] Y. Shi, K. Zhou, and N. Liu, "Engage: Explanation guided data augmentation for graph representation learning," in *Joint European*

Conference on Machine Learning and Knowledge Discovery in Databases, Cham: Springer Nature Switzerland, 2023, pp. 104–121.

[30] H. Nguyen and R. Kashef, "TS-IDS: Traffic-aware self-supervised learning for IoT Network Intrusion Detection," *Knowledge-Based Systems*, vol. 279, 4 Nov. 2023, Art. no. 110966, DOI: 10.1016/j.knosys.2023.110966.

[31] H. Zhang, Q. Wu, J. Yan, et al., "From canonical correlation analysis to self-supervised graph neural networks," *Advances in Neural Information Processing Systems*, vol. 34, pp. 76–89, 2021.

[32] M. Mirlashari and S. A. M. Rizvi, "Enhancing IoT intrusion detection system with modified E-GraphSAGE: a graph neural network approach," *Int. J. Inf. Technol.*, vol. 16, no. 4, pp. 2705–2713, Feb. 2024.

[33] E. Caville, W. W. Lo, S. Layeghy, et al., "Anomal-E: A self-supervised network intrusion detection system based on graph neural networks," *Knowl.-Based Syst.*, vol. 258, p. 110030, Dec. 2022, DOI: 10.1016/j.knosys.2022.110030.

[34] R. Xu, G. Wu, W. Wang, et al., "Applying self-supervised learning to network intrusion detection for network flows with graph neural network," *Comput. Networks*, vol. 248, p. 110495, Jun. 2024, DOI: 10.1016/j.comnet.2024.110495.

RUISHENG LI is an Professor at the School of Artificial Intelligence, Gansu University of Political Science and Law. His major research direction are network intrusion detection and malicious code analysis, intelligent information processing.

HUIMIN SHEN is a graduate student at the School of Cyberspace Security, Gansu University of Political Science and Law. Her research areas include IoT security and intrusion detection. Currently, she is primarily focused on optimizing the performance of intrusion detection models based on graph contrastive learning.

QILONG ZHANG is a graduate student at the School of Cyberspace Security, Gansu University of Political Science and Law. His research area is malicious code. His current research interests are malicious code detection based on graph neural networks.